AY26/27 Semester 1
C237 Software Application Development
Continuous Assessment 2 (CA2)

Introduction

In this project, your team will work together to **design, build, and deploy a basic web application.** This project is part of your **Continuous Assessment 2 (CA2)** and allows you to apply the core skills learned in class, including:

- **CRUD operations** with a database
- Server-side development using **Node.js and Express**
- User **authentication** and **authorization**
- Frontend rendering using **EJS**
- **Server-side JavaScript** coding

The project must be completed in a **team of 5–6 students**. Each student is expected to **contribute meaningfully** to the coding of the project and will be **assessed individually** on their assigned functionality.

Project Objective

Build a web application with:

- A **user login system**
- A resource (e.g. tasks, goals, etc.) that users can **create, view, edit, and delete**
- **At least two user roles** (e.g., admin and regular user) with different access permissions

Choosing an Application Theme

Your team should design an application that is meaningful, relevant, and based on a real-world scenario. The examples provided are for reference only. Teams are expected to adapt these ideas or develop their own application suitable for the intended users.

Some Example of Suitable Application Domains

- Event and activity management
- Home and household organization
- Travel and experience planning
- Pet ownership and care

Example:
A team interested in pet ownership may design an application to help users manage feeding schedules, appointments, expenses, and care records. Teams should determine which features best meet the needs of their intended users.

Personalisation Requirement
Your application should allow users to personalise it by managing their own information, such as expenses, activities, pet information, etc.

During the presentation, your team should be able to explain:

- Why the application theme was chosen.
- Who the intended users are.
- How the application addresses a genuine user need or problem.

Functional Expectations

The requirements below describe **what your application should achieve**, rather than prescribing a specific implementation. Teams are encouraged to choose an approach that best suits their application.

1. User Access and Identity

Your application should support user accounts and distinguish between authenticated and unauthenticated users. Protected features should only be accessible to authorised users.

2. User Roles

Your application should include at least **two user roles** with different permissions or responsibilities. Each role should have access to features appropriate to its privileges.

3. Resource Management

Your application should allow users to manage a key resource related to the application's purpose. Users should be able to create, view, update, and delete records where appropriate.

4. Finding Information

Your application should provide a way for users to locate information efficiently, such as through searching, filtering, sorting, or categorisation.

5. User Interface and Experience

Your application should provide a clear, organised, and user-friendly interface that allows users to complete tasks easily.

6. Suggested Enhancements

- Teams are encouraged to extend their application beyond the minimum requirements with **meaningful and substantial enhancements**.
- Enhancements are intended to **differentiate stronger projects** and should demonstrate technical understanding, rather than simply adding more features.
- During the presentation, teams should be able to explain:
    - Why the enhancement was introduced;
    - The user need or problem it addresses;
    - How it was implemented;
    - The challenges encountered
- A meaningful enhancement should involve substantial JavaScript, server-side logic, database interactions, or application design.

- The following are generally **not** considered meaningful enhancements:
    - Adding database fields without changing the application behaviour;
    - Hardcoded content with little or no application logic;
    - Cosmetic changes only;
    - AI-generated features that cannot be explained or justified by the team.

Example Distribution of Responsibilities (For Reference Only)

- Teams may **organise responsibilities differently based on the needs of their application**.
- Each student **MUST take ownership** of **a substantial feature** involving **JavaScript and database interactions**.
- All students must write JavaScript, not just HTML/CSS or documentation
- **app.js** should be developed collaboratively rather than by a single team member.
- The examples below illustrate how work may be divided fairly. Regardless of the distribution chosen.

Student	Example Responsibility
A	User Registration, Login and Access Control
B	Adding New Information to the System
C	Viewing and Displaying Information
D	Editing Existing Information
E	Removing Information from the System
F	Searching, Filtering or Organising Information

Team Contribution

Complete the **Team Contribution Record (Section C)** in the **CA2 Team Development Journal**.

- Record each team member's primary contributions to the project.
- This helps ensure a clear division of responsibilities.
- The completed journal must be included in your final submission.
- You may refer to the record during your presentation when explaining your individual contributions.

Submission Checklist

Deadline: 24 July 2026, 2369hrs

Where to Submit: SA3.0

Deliverables

- ✅ **Working web app**
  - **Submit the entire project folder as a .zip file which includes:**
    - The full Node.js project (excluding node_modules)
    - Your **MySQL database file** in **.sql format**
  - **Name your .zip folder using this format:**
    - TeamName_ModuleCode_ClassCode_CA2Submission.zip (e.g. TeamAlpha_C237_005_E66D_CA2Submission.zip)
    - **NOTE:** TeamName should be short and unique to your group
- ✅ **CA2 Team Development Journal (C237_CA2_Development_Journal.docx)**
- ✅ **Deployed** application (Render or similar)
- ✅ **Online MySQL database** (e.g. Azure or https://filess.io or similar)
- ✅ GitHub Repository (latest commits pushed)

Presentation

- ✅ Every team member must be able to explain and justify their assigned features, including the implementation, design decisions, and database interactions.

Important: Working code alone is insufficient. Individual marks are based on both the implementation **and** each student's ability to explain their contribution.

Use of Artificial Intelligence (AI)

Artificial Intelligence (AI) tools such as ChatGPT, Claude, or GitHub Copilot **may be used to support your learning during this assignment**.

However:

- AI should be used as a learning aid rather than a substitute for individual effort.
- Each student is responsible for **understanding all code used in the project**, including code that was generated or assisted by AI tools.
- AI-generated code should be **reviewed, tested, modified, and adapted** to suit the team's application requirements.
- Students should not rely solely on AI to complete their assigned functionality. The purpose of this project is to **demonstrate individual understanding and contribution**.
- Submissions that appear to be largely AI-generated with **minimal evidence of adaptation, contribution, or understanding** may receive **reduced marks**, even if the application is functional.
- During the presentation, lecturers may ask questions to **verify each student's understanding** of their assigned functionality.

To demonstrate your understanding, you must submit a **CA2 Team Development Journal** explaining how parts of your application were implemented.

Important Note

You will start working on this assignment as soon as possible (even **NOW!**) and are required to complete it by **24 Jul 2026, 2359hrs**:

- **Before Lesson 23 First Lesson:**
  - Complete web application and submit to **SA3.0 by 24 Jul 2026, 2359hrs.**
- **Lesson 23 & 24: Presentation**
  - Your team will deliver a ~ **20 minutes presentation**, which must include:
    - A **live demo** of your app
      - Each team member must present and explain the functionality they were responsible for developing. This includes:
        - The purpose of the feature
        - The application flow (user action -> route -> SQL query -> database -> response)
        - Key implementation decisions made
        - Challenges encountered and how they were resolved
      - Lecturers may ask follow-up questions to verify individual understanding and contribution.
    - A brief **reflection** on:
      - What you learned from this project
      - What challenges you faced and how you overcame them

Note:

- DO NOT assign a student to only do documentation or UI styling – **everyone MUST code**
- Code must reflect what was taught in class
- Students are free to be as creative as they can but do ensure that they can complete the web application within the specified timeframe.
- Students are not allowed to use full website templates that are available on the internet. Significant marks should be deducted if they are discovered doing so.
- No marks should be given to features that are hardcoded.

Penalty for late Submission

Upload and submit your assignments on time.

- DO NOT wait till the last hour or minute to submit your work.
- Make regular backups of your work.